

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №9

События

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	3
3 Описание работы программы	4
4 Результаты работы программы.....	10
5 Вывод	11
Приложение А. Листинг Программы.....	12

1 Цель работы

Изучить и реализовать механизм событий в классе, используя делегаты для уведомления пользователей о изменениях состояния объекта, а также протестировать работу событий и обработчиков в приложении, включая возможность передачи дополнительной информации о событии через аргументы.

2 Задание

Используя рассмотренные примеры, добавьте в свои классы события и протестируйте их работу. Продемонстрируйте программный код событий с полными комментариями. Потренируйтесь организовывать цепочки обработчиков события и переопределять метод, вызывающий события.

3 Описание работы программы

В данной программе реализована система событий, которая позволяет отслеживать изменения состояния компонентов автомобиля, таких как двигатель и колеса. События используются для уведомления о поломках и изменениях состояния, что делает приложение более интерактивным и отзывчивым.

В классе `Component` объявлено событие `ComponentBroken`, которое срабатывает, когда состояние компонента изменяется на "Сломано". Это событие позволяет другим частям программы реагировать на поломку компонента. В классе `Wheel` также реализовано событие `ComponentBroken`, которое уведомляет о поломке конкретного колеса. На рисунке 1 изображен класс `Wheel`.

```
Ссылка 4
public class Wheel : IComponent
{
    Ссылка 11
    public string Condition { get; set; } = "Новое"; // Состояние колеса
    Ссылка 2
    public int BrokenCount { get; set; } = 0; // Счетчик поломок колеса

    // Событие поломки колеса
    public event EventHandler ComponentBroken;

    // Метод для восстановления колеса
    Ссылка 0
    public void Repair()
    {
        // Если колесо уже новое, невозможно выполнить восстановление
        if (Condition == "Новое")
        {
            throw new InvalidOperationException("Колесо уже в хорошем состоянии.");
        }
        // Восстановление состояния колеса в "Новое"
        Condition = "Новое";
        // Генерация события о восстановлении колеса
        ComponentBroken?.Invoke(this, EventArgs.Empty);
    }

    // Метод для изменения состояния колеса
    Ссылка 4
    public void ChangeCondition(string condition)
    {
        // Проверка, что состояние колеса либо "Новое", либо "Сломано"
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        // Изменение состояния колеса
        Condition = condition;

        // Если колесо сломано, увеличиваем счетчик поломок и генерируем событие
        if (Condition == "Сломано")
        {
            BrokenCount++; // Увеличиваем счетчик поломок
            ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события при поломке
        }
    }

    // Метод для получения информации о колесе
    Ссылка 0
    public string GetInfo(bool showDetailed)
    {
        // Базовая информация о состоянии колеса
        string info = $"Колесо состояние: {Condition}";

        if (showDetailed)
        {
            // Если запрашиваются подробности, добавляем информацию о количестве поломок
            info += $"\\nКоличество поломок: {BrokenCount}";
        }

        return info; // Возвращаем итоговую информацию
    }
}
```

Рисунок 1 – Класс `Wheel`

В конструкторе класса Car происходит подписка на события поломки двигателя и колес. Каждый раз, когда создается новый объект Wheel, он подписывается на событие ComponentBroken, что позволяет автомобилю отслеживать состояние всех своих колес. Двигатель также подписывается на событие поломки, что позволяет автомобилю реагировать на его состояние. На рисунке 2 изображен класс IComponent.

```
Ссылка 1
public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    Ссылка 11
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    Ссылка 4
    void ChangeCondition(string condition);
}
```

Рисунок 2 – Интерфейс IComponent

Метод OnComponentBroken в классе Car обрабатывает события поломки. Когда происходит поломка двигателя или колеса, этот метод выводит сообщение в консоль, информируя пользователя о том, какой компонент сломался. Это позволяет пользователю быстро реагировать на проблемы с автомобилем. В зависимости от того, какой компонент вызвал событие, метод определяет, нужно ли выводить сообщение о поломке двигателя или колеса. На рисунке 3 изображен класс Component.

```
Ссылка 3
public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    Ссылка 9
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    Ссылка 0
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    Ссылка 1
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
    public event EventHandler ComponentBroken;
}
```

Рисунок 3 – Класс Component

Система событий позволяет легко добавлять новые компоненты и обрабатывать их состояния, не изменяя основную логику программы. Например, если в будущем будет добавлен новый компонент, его можно будет легко интегрировать в систему событий, подписав его на соответствующие события. В классе Info, который наследует от класса Car, можно переопределить методы, связанные с событиями, чтобы предоставить более детализированную информацию о состоянии автомобиля, если это необходимо. На рисунке 4 изображен класс Info.

```
Слайд 3
public sealed class Info : Car
{
    // Конструктор для класса Info, который вызывает конструктор базового класса
    Слайд 1
    public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
        : base(name, horsepower, fuelConsumption, fuelTankCapacity)
    {
    }

    // Переопределяем метод получения информации о машине
    Слайд 3
    public override string GetCarInfo(bool showDetailed = false)
    {
        string info = $"Название: {Name}\n" +
            $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
            $"Пробег: {Mileage} км\n" +
            $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
            $"Расход топлива: {FuelConsumption} л/100 км\n" +
            $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
            $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

        if (showDetailed)
        {
            info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
            for (int i = 0; i < Wheels.Length; i++)
            {
                info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
            }
        }

        return info;
    }
}
```

Рисунок 4 – Класс Info

Таким образом, использование событий в данной программе позволяет эффективно управлять состоянием автомобиля и его компонентов, обеспечивая пользователю актуальную информацию о состоянии транспортного средства и позволяя быстро реагировать на возникающие проблемы.

На рисунке 5 изображена диаграмма классов программы.

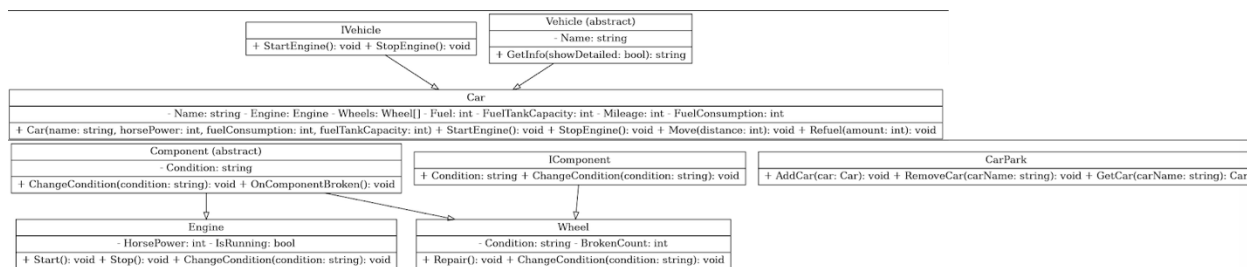


Рисунок 5 – Диаграмма классов

На рисунке 6 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area contains several input fields and buttons. On the left, there are four labels with corresponding text boxes: "Введите имя авто:", "Введите мощность авто:", "Введите расход авто:", and "Введите бак авто:". Below these are two buttons: "Создать авто" and "Удалить авто". At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать". In the center, there is a large empty rectangular box. On the right side, there are four buttons stacked vertically: "Заправка", "Передвижение", "Запустить", and "Выключить". Below these buttons is a checkbox labeled "Высокая скорость" and a button labeled "Заменить".

Рисунок 6 – Оформление окна Windows Forms

4 Результаты работы программы с примерами

На рисунке 7 представлен результат работы программы показа состояния компонента.

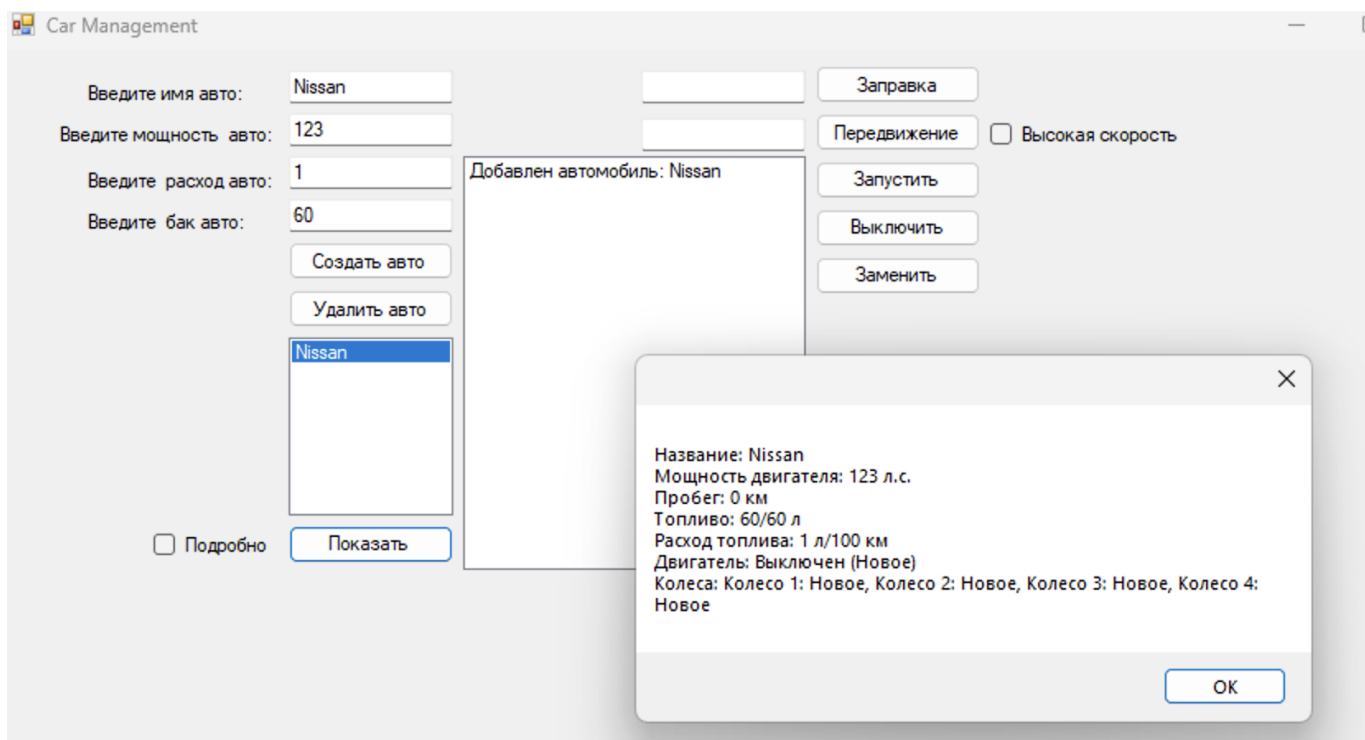


Рисунок 7 – Результат работы программы

5 Вывод

В результате реализации системы управления автомобилем с использованием событий, приложение демонстрирует высокую степень интерактивности и отзывчивости. События позволяют эффективно отслеживать изменения состояния компонентов автомобиля, таких как двигатель и колеса, и уведомлять пользователя о возникших проблемах. Это обеспечивает возможность быстрого реагирования на поломки и улучшает общий пользовательский опыт.

Система событий также способствует гибкости и расширяемости приложения, позволяя легко добавлять новые компоненты и обрабатывать их состояния без необходимости значительных изменений в основной логике программы. Переопределение методов и подписка на события обеспечивают возможность кастомизации поведения приложения в зависимости от конкретных требований.

Таким образом, использование событий в данном приложении не только улучшает управление состоянием автомобиля, но и демонстрирует принципы объектно-ориентированного программирования, такие как инкапсуляция, наследование и полиморфизм, что делает приложение более структурированным и удобным для дальнейшего развития.

Приложение А. Листинг Программы

Файл «Component»

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
```

Файл «IComponent»

```
using System;

public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    void ChangeCondition(string condition);
}
```

Файл «Info»

```
using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {

```

```

// Конструктор для класса Info, который вызывает конструктор базового класса
public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
    : base(name, horsepower, fuelConsumption, fuelTankCapacity)
{
}

// Переопределяем метод получения информации о машине
public override string GetCarInfo(bool showDetailed = false)
{
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}
}

```